

Week 4. Homework 2. SQL-II

Submission Guidelines

- **Dataset:** You must use the provided *IMDB.sqlite* dataset to answer all questions.
- **Code and Output:** Copy your code and attach screenshots of your outputs under each question.
- **Submission Format:** Each student is required to push one Word or PDF file to a designated GitHub repository you create specifically for this week's assignment.
- **Repository Name:** Use the format *Week 4-HW2-LastName*, where *LastName* is your last name.
- **Code Quality:** Ensure your code is organized for easy understanding.
- **AI Usage:** You should make a serious attempt at each problem prior to using an LLM. Remember, you may be tested on SQL knowledge without assistance in a technical interview. If prompting an LLM, I suggest telling it your starting point, your thought process, and asking it to guide you. I've done this learning SQL and it is helpful and I learned more that way. Disclose your usage at the end of the file; you will not be penalized regardless, but this give me some information.

Questions

Date-Based Queries

1. Find the First and Last Movie Released Each Year

Retrieve the first and last movie released in each year. Display the year, first movie title, and last movie title.

```
--Question 1
SELECT strftime('%Y', release_date) AS year,
(
SELECT title
FROM movies AS first_movie
WHERE strftime('%Y', first_movie.release_date) = strftime('%Y', year_movies.release_date)
ORDER BY release_date ASC
LIMIT 1) AS first_movie,
(
SELECT title
FROM movies AS last_movie
WHERE strftime('%Y', last_movie.release_date) = strftime('%Y', year_movies.release_date)
ORDER BY release_date DESC
LIMIT 1) AS last_movie
FROM movies AS year_movies
GROUP BY year
ORDER BY year;
```

	year	first_movie	last_movie	
1	1916	Intolerance	Intolerance	
2	1926	The Big Parade	The Big Parade	
3	1927	Metropolis	Metropolis	
4	1929	Pandora's Box	The Broadway Melody	
5	1930	Hell's Angels	Hell's Angels	
6	1932	A Farewell to Arms	A Farewell to Arms	
7	1933	42nd Street	She Done Him Wrong	
8	1934	It Happened One Night	It Happened One Night	
9	1935	Top Hat	Top Hat	
10	1936	Modern Times	The Charge of the Light Brigade	

2. Total Revenue Generated Each Year

Calculate the total revenue generated by movies released in each year. Display the year and total revenue, ordered by revenue in descending order.

```
--Question 2
SELECT strftime('%Y', release_date) AS year,
SUM(revenue) AS total_revenue
FROM movies
GROUP BY year
ORDER BY total_revenue DESC;
```

	year	total_revenue
1	2012	24141710246
2	2014	24120490589
3	2013	23411493295
4	2015	22775024221
5	2009	21072651506
6	2011	20516921160
7	2010	20348574768
8	2008	18145379803

3. Movies Released in the First Half of the Year

Find all movies that were released between January and June. Display the movie title, release date, and director name.

```
--Question 3
SELECT title, release_date, name
FROM movies
JOIN directors ON movies.director_id = directors.id
WHERE strftime('%m', release_date) BETWEEN '01' AND '06'
ORDER BY release_date ASC;
```

	title	release_date	name
1	Metropolis	1927-01-10	Fritz Lang
2	Pandora's Box	1929-01-30	G.W. Pabst
3	The Broadway Melody	1929-02-08	Harry Beaumont
4	42nd Street	1933-02-02	Lloyd Bacon
5	She Done Him Wrong	1933-02-09	Lowell Sherman
6	It Happened One Night	1934-02-22	Frank Capra
7	Modern Times	1936-02-05	Charlie Chaplin
8	Alexander's Ragtime Band	1938-05-24	Henry King

4. Number of Movies Released on Weekends

Count the total number of movies released on a weekend (Saturday or Sunday). Use SQLite's strftime('%w') to determine the day of the week (0 = Sunday, 6 = Saturday).

```
--Question 4
SELECT COUNT(*) AS weekend_movies
FROM movies
WHERE strftime('%w', release_date) IN ('0', '6');
```

	weekend_movies
1	451

CASE Statement Queries

5. Categorize Directors Based on Total Revenue

```
--Question 5
SELECT directors.name,
SUM(movies.revenue) AS total_revenue,
CASE
WHEN SUM(movies.revenue) > 1000000000 THEN 'High Revenue'
WHEN SUM(movies.revenue) >= 500000000 THEN 'Moderate Revenue'
ELSE 'Low Revenue'
END AS revenue_category

FROM directors
JOIN movies ON directors.id = movies.director_id
GROUP BY directors.name
```

	name	total_revenue	revenue_category	
1	Aaron Hann	0	Low Revenue	
2	Aaron Schneider	0	Low Revenue	
3	Abel Ferrara	1227324	Low Revenue	
4	Adam Brooks	55447968	Low Revenue	
5	Adam Carolla	0	Low Revenue	
6	Adam Goldberg	0	Low Revenue	
7	Adam Green	0	Low Revenue	
8	Adam Jay Epstein	0	Low Revenue	
9	Adam Marcus	15938065	Low Revenue	
10	Adam McKay	859076455	Moderate Revenue	

6. Movies Based on Popularity and Revenue

Classify movies into the following categories based on popularity and revenue:

- Blockbuster if popularity > 100 and revenue > \$500 million
- Hit if popularity > 50 and revenue between \$100 million and \$500 million
- Average for all other movies

```
--Question 6
SELECT title, popularity, revenue,
CASE
WHEN popularity > 100 AND revenue > 500000000 THEN 'Blockbuster'
WHEN popularity > 50 AND revenue BETWEEN 100000000 AND 500000000 THEN 'Hit'
ELSE 'Average'
END AS popularity_type
FROM movies
```

	title	popularity	revenue	popularity_type
1	Avatar	180	2787968087	Blockbuster
2	Pirates of the Caribbean: At World's End	139	961000000	Blockbuster
3	Spectre	107	880674609	Blockbuster
4	The Dark Knight Rises	112	1084939099	Blockbuster
5	John Carter	43	284139100	Average
6	Spider-Man 3	115	890871628	Blockbuster
7	Tangled	48	591794936	Average
8	Avengers: Age of Ultron	134	1405403694	Blockbuster
9	Harry Potter and the Half-Blood Prince	98	933969197	Average
10	Batman v Superman: Dawn of Justice	155	673260194	Blockbuster

7. Directors with Critically Acclaimed and Poorly Rated Movies

List directors and count how many of their movies fall into these categories:

- Critically Acclaimed if vote_average >= 8
- Poorly Rated if vote_average < 5

```
--Question 7
SELECT directors.name,
COUNT(
CASE
WHEN vote_average >= 8 THEN title END) AS 'Critically Acclaimed',
COUNT(
CASE
WHEN vote_average < 5 THEN title END) AS 'Poorly Rated'
FROM directors
JOIN movies ON directors.id = movies.director_id
GROUP BY directors.name
```

	name	Critically Acclaimed	Poorly Rated
2	Aaron Schneider	0	0
3	Abel Ferrara	0	0
4	Adam Brooks	0	0
5	Adam Carolla	0	0
6	Adam Goldberg	0	0
7	Adam Green	0	0
8	Adam Jay Epstein	0	1
9	Adam Marcus	0	1
10	Adam McKay	0	0
11	Adam Rapp	0	0
12	Adam Rifkin	0	0
13	Adam Shankman	0	0
14	Adrian Lyne	0	0
15	Adrienne Shelly	0	0
16	Agnieszka Holland	0	0
17	Agnieszka Wajtowicz-Wosloo	0	0
18	Agustin Diaz Yanes	0	0
19	Aki Kaurismäki	0	0
20	Akira Kurosawa	1	0

- **AI Usage:** For most of the questions I attempted to do them by myself without the use of AI, most of the questions were straightforward especially after studying the slides and practicing the examples in the sql slides given by the professor. Question 1 was the most challenging for me and I attempted to do it by myself and when I got stuck or ran across an error when executing the sql line I asked AI to assist me and teach me first and explain the use for each function.